SC3000 Artificial Intelligence

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Introduction: Intelligent Agents

> *"An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators."* — Stuart Russell and Peter Norvig. This chapter builds that idea from everyday intuition to a precise engineering framework.

From zero: no AI background assumed. We start from what you already know—thermostats, phone apps, people—and build up *agent*, *rationality*, *PEAS*, and a map of AI history before any search or learning algorithm. Every term is defined on first use, picture first, then formula.

Learning Outcomes

After this chapter you will be able to:

(L1) define *agent*, *percept*, *action*, and *agent function* and map them to sensors and actuators;

(L2) state the *rational agent* criterion and distinguish rationality from omniscience and perfection;

(L3) specify a task environment with the *PEAS* framework (Performance, Environment, Actuators, Sensors);

(L4) classify environments along PEAS dimensions (observable, deterministic, episodic, etc.);

(L5) recount major milestones of AI history and why each shifted the field.

1.1 What Is an Agent?

Intuition. Look at a room thermostat. It *senses* temperature, compares it to a set-point, and *acts* by switching the heater on or off. You do the same: eyes and ears sense, hands and voice act. The thermostat, a chatbot, a self-driving car, and you all fit one pattern—a loop with the world.

Formal definition. An *agent* perceives its *environment* through *sensors* and acts through *actuators*. At time t it receives a *percept* p_t; the full history $p_1, \ldots, p_t$ is the *percept sequence*. The agent's behaviour is an *agent*

function

$$f : \mathcal{P}^* \to \mathcal{A}, \qquad f(p_1, \ldots, p_t) = a_t,$$

mapping every percept sequence to an action. The *agent program* is the concrete implementation of f on an architecture (hardware + software).

Example. Vacuum world: two squares A and B, dirt appears at random. Percepts are $[\mathit{location}, \mathit{dirt?}] \in \{A, B\} \times \{\mathit{Dirty}, \mathit{Clean}\}$. Actions are $\{\mathit{Left}, \mathit{Right}, \mathit{Suck}, \mathit{NoOp}\}$. The sequence $[A, \mathit{Dirty}], [A, \mathit{Clean}], [B, \mathit{Dirty}]$ maps to $a_3 = \mathit{Suck}$ under a sensible f.

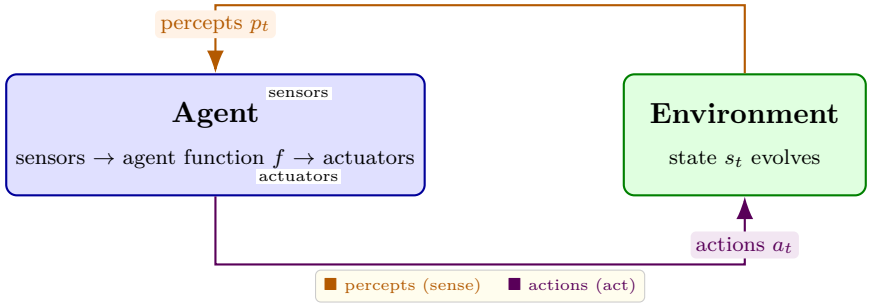

Figure 1.1: Agent–environment loop. The agent receives percepts through sensors, selects $a_t = f(p_1, \ldots, p_t)$, and modifies the environment through actuators; the environment updates to s_{t+1} and emits the next percept.

1.2 Rationality and Performance

Intuition. A good thermostat is not one that *knows* the weather tomorrow; it is one that does the *best it can* with the sensor it has and the energy bill you care about. Rationality is about expected outcome, not psychic perfection.

Formal criterion. Fix a *performance measure* U that scores environment histories (e.g., $+10$ per clean square, -1 per move). A *rational agent* acts to maximise *expected* U given the percept sequence, its built-in knowledge, and the actions it can execute:

$$a_t^* = \arg\max_{a \in \mathcal{A}} \mathbb{E}\left[U \mid p_{1:t}, \text{ knowledge}, a\right].$$

Rational $\neq$ omniscient (which knows the true outcome) and $\neq$ clairvoyant (which knows all future percepts). Rationality also requires *autonomy*: the agent should learn from experience rather than rely solely on designer priors.

Example. Crossing a road: performance is safety + speed. An omniscient agent would know no car comes and stride blindly; a rational agent looks both ways, waits for evidence, and then crosses—even though looking costs a second. If it never learns that a certain crossing is usually empty, it lacks autonomy.

1.3 Specifying Tasks: The PEAS Framework

Intuition. "Build me an AI for hospitals" is not a specification. Four questions make it one: How will we *measure* success? What *world* will it face? What can it *do*? What can it *see*?

Formal framework. Every task environment is described by **PEAS**:

P — Performance measure The objective criterion U the rational agent maximises.

E — Environment The world, other agents, and dynamics the agent interacts with.

A — Actuators The actions/channels available to the agent.

S — Sensors The percepts/inputs through which the agent observes.

Table 1.1 fills PEAS for four representative agents. Designing PEAS *before* choosing an algorithm forces clarity about the trade-off you actually care about.

Agent	Performance	Environment	Actuators	Sensors
Self-driving taxi	Safety, time, comfort, profit	Roads, traffic, passengers	Steering, brake, display	Cameras, lidar, GPS, mic
Medical diagnosis	Accuracy, cost, explanation	Patient, hospital, diseases	Screen, report, referral	Symptoms, labs, images
Part-picking robot	% parts correctly binned	Conveyor, bins, other arms	Arm, gripper, sorter	Camera, force sensor
Chatbot tutor	Learning gain, satisfaction	Student, curriculum, network	Text, speech, exercises	Keystrokes, answers, clicks

Table 1.1: PEAS descriptions make the design problem explicit. Changing any column (e.g., adding a sensor) changes the rational policy.

Environments are also characterised along six dimensions: fully vs. partially *observable*, single vs. multi *agent*, *deterministic* vs. stochastic, *episodic* vs. sequential, *static* vs. dynamic, *discrete* vs. continuous. A taxi is partially observable, multi-agent, stochastic, sequential, dynamic, continuous; a crossword puzzle is fully observable, single-agent, deterministic, sequential, static, discrete.

Listing 1.1: PEAS as a data structure: specifying a task before coding the agent.

```python
from dataclasses import dataclass

@dataclass
class PEAS:
    performance: str
    environment: str
    actuators: list
    sensors: list

taxi = PEAS(
    performance="minimise time + maximise safety + profit",
    environment="roads, traffic, pedestrians, weather",
    actuators=["steer", "accelerate", "brake", "horn", "display"],
    sensors=["camera", "lidar", "GPS", "microphone", "odometer"]
)

def is_rational(action, percept_seq, peas: PEAS) -> bool:
    """Stub: compare action against expected performance measure."""
    # In Ch.2-5 this becomes search; in Ch.8 it becomes learning.
```

```
    return True  # placeholder for expected-utility test
```

1.4 A Brief History of AI

Intuition. AI did not appear with ChatGPT. It is an 80-year arc of big claims, winters, and quiet engineering that suddenly looked like magic when scale and data caught up.

Timeline (formal milestones). Five eras are conventional: *foundations* (1943–1955), *symbolic AI* (1956–1974), *knowledge and expert systems* (1969–1986), *statistical learning* (1986–2011), *deep learning and foundation models* (2012–present).

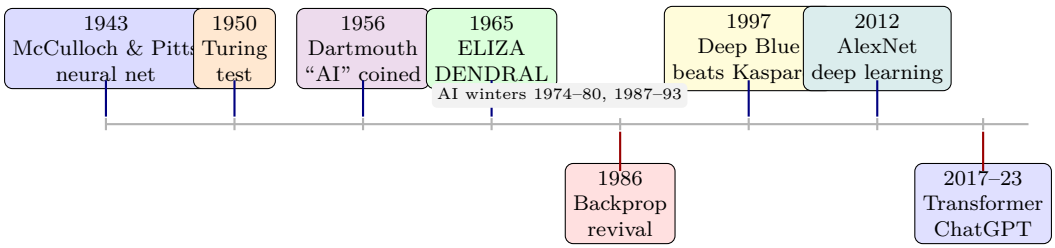

Figure 1.2: Condensed history of AI. Colour groups eras: blue/orange = foundations, violet/green = symbolic, red/yellow = knowledge and competition, teal/blue = deep learning era. Winters mark funding contractions after over-promising.

Why it matters: each era changed what an agent could be. Logic gave us provably correct agents but brittle ones; expert systems added human knowledge but were costly to maintain; statistical learning let agents handle uncertainty; deep networks let them learn their own features from raw sensors.

Listing 1.2: A simple reflex agent for the vacuum world. No memory beyond the current percept.

```
def simple_reflex_vacuum(percept):
    """percept = (location, status) where status in {'Dirty','Clean'}.
       Returns an action in {'Suck','Right','Left','NoOp'}."""
    location, status = percept
    if status == 'Dirty':
        return 'Suck'
    # otherwise move toward the other square
    if location == 'A':
        return 'Right'
    else:
        return 'Left'

# Simulation stub: percept sequence -> action
trace = [('A','Dirty'), ('A','Clean'), ('B','Dirty')]
for p in trace:
    print(p, "->", simple_reflex_vacuum(p))
```

Reflex agents work when the current percept suffices (fully observable, episodic). When history matters, we need model-based, goal-based, or utility-based agents—the subject of later chapters.

1.5 Chapter Notes

Further reading: Russell & Norvig Ch. 1–2 for the canonical PEAS and agent taxonomy; Nilsson (1998) for the historical arc; Turing (1950) for the imitation game. The PEAS table pattern is due to Russell & Norvig and is the standard scoping tool in NTU SC3000 design exercises.

1.6 Exercises

E1.1 **Agent function vs. program.** Give the agent function f for a 2-state thermostat (percepts $\{Cold, Hot\}$, actions $\{On, Off\}$) that turns heat on when cold and off when hot. Write the program as a Python dictionary lookup. Why can two different programs implement the same function?

E1.2 **Rationality, not omniscience.** A vacuum agent is penalised -1 per move and $+10$ per clean. It does not know whether dirt will reappear. Compare a rational agent that maximises expected score against an "omniscient" agent that knows the future dirt schedule. Give a percept sequence where they choose differently and compute expected vs. actual scores.

E1.3 **PEAS design.** Specify PEAS for (a) an AI tutor for SC2001 algorithms and (b) an automated warehouse picker. Fill one row of Table 1.1 for each. For each, state whether the environment is fully/partially observable, deterministic/stochastic, and episodic/sequential, and justify in one sentence per dimension.

E1.4 **Reflex limits.** Show that no simple reflex agent (mapping current percept only) can be rational in a partially observable vacuum world where the agent cannot sense its location (percept is only *Dirty*/*Clean*). Propose a minimal memory extension that restores rationality and sketch its code.

E1.5 **History and winters.** Pick two milestones from Fig. 1.2 (one before 1980, one after 2010). For each, name the bottleneck it removed and one new limitation it introduced that led to the next winter or shift.

Chapter 2

Uninformed Search: Breadth, Depth and Cost

"Search is where intelligence begins." — An agent that can imagine futures, try them mentally, and pick the best trip has already shown intelligence. This chapter builds *blind* search from zero: no heuristic, just the map and a frontier of possibilities.

From zero: we build here, no prior AI needed. We define state, action, and search tree from scratch—with pictures before formulas. No heuristic or graph theory assumed. Later chapters (informed search, CSPs) reuse what is built here.

Learning Outcomes

After this chapter you will be able to:

(L1) formalise any problem as a *search problem* (S, A, T, s_0, G, c);

(L2) distinguish tree search vs. graph search and explain the explored set;

(L3) describe the *frontier* (open list) and how its discipline defines BFS, DFS, UCS, IDS;

(L4) analyse completeness, optimality, time and space of each blind strategy;

(L5) implement BFS and DFS/IDS and choose the right blind algorithm for a task.

2.1 What Is Search? Problem Formulation

Intuition first. Imagine you are lost in a subway maze with a paper map. You mark your current station, list every train you could take, and keep a *to-try* list of stations you have seen but not yet explored. Search is exactly this mental simulation.

Definition 2.1 (Search problem)**.** A search problem is a tuple $(S, A, \text{Succ}, s_0, G, c)$ where S is the set of *states*,

A the set of *actions*, $\mathrm{Succ}(s) \subseteq S$ the successors of s, $s_0 \in S$ the *initial state*, $G \subseteq S$ the *goal test* (often a predicate $\mathrm{Goal}(s)$), and $c(s, a, s') \geq 0$ the step cost.

A *solution* is a path $s_0 \to s_1 \to \cdots \to s_k$ with $s_k \in G$; its cost is $\sum_i c(s_i, a_i, s_{i+1})$. An *optimal* solution has minimum cost. The *state space* is the directed graph (S, E) with $E = \{(s, s') : s' \in \mathrm{Succ}(s)\}$—distinct from the *search tree* we grow while exploring.

Example 2.1 (Romania road trip (Russell–Norvig))**.** S = cities, s_0 = Arad, $G = \{\text{Bucharest}\}$, $\mathrm{Succ}(\text{Arad}) = \{\text{Sibiu}, \text{Timisoara}, \text{Zerind}\}$, c = road distance. Finding a route Arad $\to$ Bucharest is a shortest-path search problem.

The agent needs a *node* data structure: $\langle s, \text{parent}, \text{action}, g \rangle$ where g is path cost from s_0 to s. Expanding a node generates its successors. The set of generated-but-unexpanded nodes is the *frontier* (or open list)—the boundary between explored and unseen. Which node we pick from the frontier *is* the search strategy.

2.2 Tree Search vs. Graph Search and the Frontier

In *tree search* we never remember where we have been: the same state can be generated along many paths (imagine revisiting Arad via two routes). In *graph search* we maintain an *explored* (closed) set of expanded states and discard any new node whose state is already explored or in the frontier—pruning loops and redundant paths.

Graph search is complete and more efficient when the state space has cycles or many duplicate paths; tree search suffices (and saves memory) only when the state space is a tree. Prices: graph search needs $O(|S|)$ memory for the closed set.

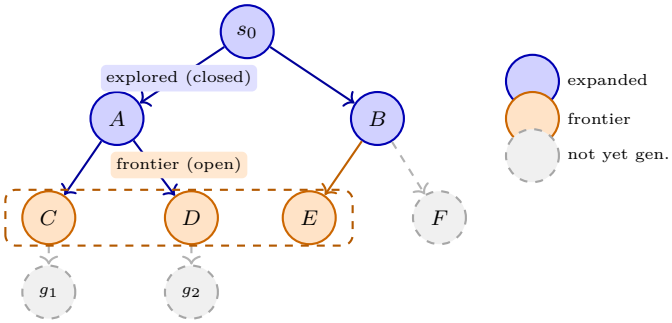

Figure 2.1: Search tree vs. state space. Blue = explored (already expanded), orange solid = frontier (generated, unexpanded), dashed grey = not yet generated. Graph search would prune a successor already in blue/orange.

General graph search loops: pop a node from *frontier*; test goal; add state to *explored*; expand and insert unseen successors into frontier. The frontier's queueing discipline determines the algorithm.

2.3 Breadth-First and Depth-First Search

BFS uses a FIFO queue (frontier = queue): expand shallowest nodes first, level by level. *DFS* uses a LIFO stack: plunge down one branch before backtracking. Fig. 2.2 shows the order in which nodes are expanded on the same tree.

If branching factor is b, shallowest goal depth is d, and maximum depth is m: BFS is *complete* (finds a goal if one exists) and *optimal* when all step costs are equal (first goal found is shallowest). It costs $O(b^d)$ time and $O(b^d)$ memory—memory is the bottleneck. DFS needs only $O(bm)$ memory (the path plus siblings) and $O(b^m)$ time, but is *neither complete* (infinite branches) nor optimal in general; graph-search with cycle checking restores completeness on finite spaces.

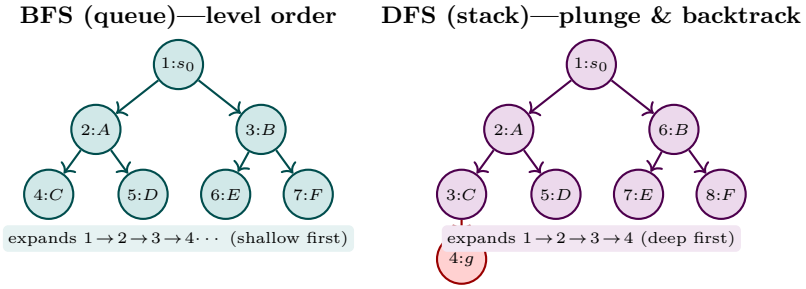

Figure 2.2: BFS vs. DFS expansion order (numbers) on the same tree. BFS finds shallow goals first; DFS dives to depth m with far less memory. Orange frontier in Fig. 2.1 becomes a queue (BFS) or stack (DFS).

Listing 2.1: Breadth-first search (graph search, FIFO frontier). Returns path or None.

```python
from collections import deque

def bfs(problem):
    """problem: object with initial, is_goal(s), successors(s)->list[s], cost(s,s')."""
    start = problem.initial
    if problem.is_goal(start):
        return [start]
    frontier = deque([[start]])          # queue of paths
    explored = {start}
    while frontier:
        path = frontier.popleft()        # FIFO -> BFS
        s = path[-1]
        for ns in problem.successors(s):
            if ns not in explored:
                if problem.is_goal(ns):
                    return path + [ns]
                explored.add(ns)
                frontier.append(path + [ns])
    return None
```

2.4 Uniform-Cost Search and Iterative Deepening

When $c(s, a, s')$ varies, shallowest $\neq$ cheapest. *Uniform-cost search (UCS)* is BFS with costs: frontier is a priority queue ordered by $g(n)$ (path cost from s_0). It expands nodes in increasing g—exactly Dijkstra's algorithm. UCS is complete and optimal when $c \geq \epsilon > 0$, with $O(b^{1+C^*/\epsilon})$ time where C^* is optimal cost.

Iterative deepening search (IDS) runs DFS with depth limit $l = 0, 1, 2, \ldots$ until a goal is found. Each iteration re-expands upper levels, but the overhead is small: $\sum_{i=0}^{d} b^i/b^d \approx b/(b-1)$. IDS is complete and optimal for uniform costs, with BFS's optimality but DFS's $O(bd)$ memory—the preferred blind method when b is moderate and d unknown. The depth-limited subroutine is just DFS that cuts off at depth l.

Listing 2.2: Depth-limited DFS and iterative deepening (graph search with depth cap).

```python
def dls(problem, limit):
    """Depth-limited DFS; returns path or None. Uses stack (LIFO)."""
    frontier = [([problem.initial], 0)]  # (path, depth)
    explored = set()
    while frontier:
        path, d = frontier.pop()         # LIFO -> DFS
        s = path[-1]
        if problem.is_goal(s):
            return path
        if d < limit and s not in explored:
            explored.add(s)
            for ns in reversed(problem.successors(s)):
                if ns not in explored:
                    frontier.append((path + [ns], d + 1))
    return None

def ids(problem, max_depth=50):
    for lim in range(max_depth + 1):
        res = dls(problem, lim)
        if res is not None:
            return res
    return None
```

Criterion	BFS	DFS (graph)	UCS	IDS
Complete	Yes	Yes*	Yes ($c \geq \epsilon$)	Yes
Optimal (uniform/equal cost)	Yes	No	Yes (any c)	Yes
Time	$O(b^d)$	$O(b^m)$	$O(b^{1+C^*/\epsilon})$	$O(b^d)$
Space	$O(b^d)$	$O(bm)$	$O(b^{1+C^*/\epsilon})$	$O(bd)$
Frontier	FIFO queue	LIFO stack	priority by g	LIFO + depth limit

Table 2.1: Blind search at a glance. b = branching, d = goal depth, m = max depth, C^* = optimal cost. *Graph-DFS complete on finite spaces; tree-DFS can cycle forever.

2.5 Chapter Notes

Blind search was formalised in early AI (Newell & Simon, 1963; Nilsson, 1971). BFS/DFS are graph traversals from the 19th century (Trémaux, Tarry); UCS is Dijkstra (1959) recast for AI. IDS (Korf, 1985) shows that repetition can buy memory. All strategies here are *uninformed*—they use no estimate of distance to goal; Ch. 3 adds heuristics (A^*) to guide the frontier.

2.6 Exercises

E2.1 **Formulation.** Formalise 8-puzzle as $(S, A, \text{Succ}, s_0, G, c)$. What is $|S|$ (reachable)? Give b (avg branching) and why Manhattan distance is not part of this chapter's blind search.

E2.2 **Tree vs. graph.** On a grid with 4-neighbour moves, start $(0, 0)$, goal $(2, 1)$, no obstacles. Draw the search tree to depth 2. How many tree nodes vs. distinct states? Which duplicate states does graph search prune

at depth 2?

E2.3 **BFS/DFS/UCS by hand.** Graph: $s_0 \to A(2), B(1); A \to G(2); B \to A(1), G(5)$ where (c) is edge cost. Give expansion order and returned path for (a) BFS, (b) DFS (alphabetic successor order), (c) UCS. Which are optimal?

E2.4 **IDS cost.** Prove IDS expands $N_{\text{IDS}} = \sum_{i=0}^{d}(d+1-i)b^i$ nodes and $N_{\text{BFS}} = \sum_{i=0}^{d} b^i$. Show $N_{\text{IDS}}/N_{\text{BFS}} \to b/(b-1)$ as d grows, and evaluate for $b = 10, d = 6$.

E2.5 **Programming.** Implement `bfs` and `ids` above on a Romania map (8–10 cities). Compare nodes expanded and max frontier size for Arad→Bucharest. Verify BFS expands $\Theta(b^d)$ nodes while IDS uses $O(bd)$ frontier memory.

Chapter 3

Informed Search

Uninformed search explores blindly; informed search guesses wisely
— guided by a heuristic that estimates how far remains to the goal.

Learning outcomes. After this chapter you will be able to: (1) define g, h, f and explain $f = g + h$; (2) distinguish greedy best-first from A*; (3) test admissibility vs. consistency; (4) prove A* optimality; (5) run IDA* and choose a heuristic by dominance.

Prerequisites: uninformed search (Ch. 2), priority queues, and big-O. We build from zero: no prior heuristic knowledge assumed.

3.1 From Blind to Guided Search

Uninformed methods (BFS, DFS, UCS) use only the path cost $g(n)$ — the cheapest known cost from start S to node n. They cannot tell whether n is promising. *Informed* (heuristic) search adds a guess $h(n) \approx h^*(n)$, the true cheapest cost $n \to$ goal. The art is choosing h that is informative yet never misleading.

Definition 3.1 (Heuristic function)**.** A heuristic $h : \mathcal{S} \to \mathbb{R}_{\geq 0}$ maps each state to a non-negative estimate of the cheapest cost to the nearest goal, with $h(G) = 0$ for every goal G.

Intuition: on a road map, *straight-line distance* to Bucharest underestimates any real route — it is optimistic and therefore safe. Manhattan distance on a 4-neighbour grid has the same property when each move costs 1. Relaxing the problem (ignore roads, ignore obstacles) is the standard way to invent admissible heuristics.

3.2 Greedy Best-First Search

Greedy best-first expands the node that *looks* closest to goal:

$$f(n) = h(n).$$

It picks the smallest h, ignoring how far we have already travelled. It is fast and often finds a solution quickly, but it is neither complete (in infinite spaces) nor optimal.

Example 3.1 (Greedy trap)**.** On the Romania map, straight-line h makes greedy go Arad → Sibiu → Fagaras → Bucharest (450 km) even though Arad → Sibiu → Rimnicu → Pitesti → Bucharest (418 km) is cheaper. Greedy never looks back at g.

UCS is the opposite extreme: $f(n) = g(n)$, optimal but explores equally in all directions. A* combines both.

3.3 A* Search: $f(n) = g(n) + h(n)$

Definition 3.2 (A* evaluation)**.** For node n, let $g(n)$ be the cheapest known cost $S \to n$ and $h(n)$ the heuristic. A* orders its frontier (priority queue) by

$$f(n) = g(n) + h(n),$$

an estimate of the total cost of the best solution constrained to go through n.

At each step A* pops the smallest-f node, tests for goal, expands successors, and pushes them with their f values. With a min-heap this is $O(\log |\text{frontier}|)$ per push/pop. If h never overestimates, the first goal popped is optimal.

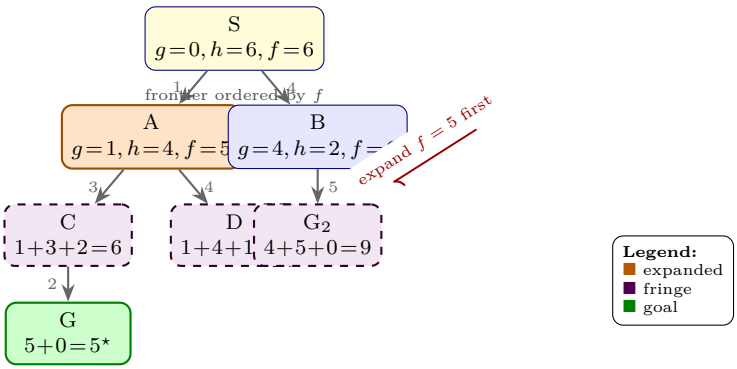

Figure 3.1: A* search tree. Each node shows $g + h = f$. A* expands smallest f first; the goal via A–C ($f = 5$) is found before the costlier B route ($f = 9$).

3.4 Admissibility and Consistency

Definition 3.3 (Admissibility)**.** h is *admissible* iff it never overestimates: $0 \le h(n) \le h^*(n)$ for all n, where $h^*(n)$ is the true cheapest cost $n \to$ goal.

Definition 3.4 (Consistency / Monotonicity)**.** h is *consistent* iff for every edge $n \to n'$ with cost $c(n, n')$,

$$h(n) \le c(n, n') + h(n') \quad \text{(triangle inequality)},$$

and $h(G) = 0$. Consistency implies admissibility; the converse is false.

Consistency guarantees f is non-decreasing along any path, so A* with graph search never needs to reopen a closed node. With tree search, admissibility alone suffices for optimality; with graph search, consistency is needed (otherwise a better g to a closed node may appear later).

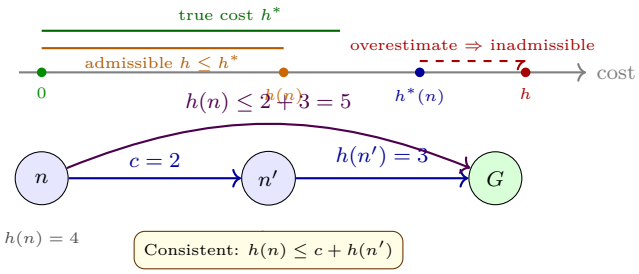

Figure 3.2: Top: admissibility — h must lie at or below h^*. Bottom: consistency as a triangle inequality on every edge.

Optimality sketch. With admissible h, any suboptimal goal G_2 has $f(G_2) = g(G_2) > g^*$. An optimal goal G^* on the frontier satisfies $f(G^*) = g^* \le f(G_2)$, so A* pops G^* first. Dominance matters: if $h_2 \ge h_1$ everywhere, A* with h_2 expands a subset of the nodes of h_1 (up to tie-breaking).

3.5 IDA*: Memory-Bounded Optimal Search

A* keeps the entire frontier ($O(b^d)$ memory, b branching, d depth). *Iterative-Deepening A** (IDA*) runs depth-first search limited by an f-threshold, increasing it to the minimum f that exceeded it:

$$\text{threshold}_{k+1} = \min\{f(n) \mid f(n) > \text{threshold}_k\}.$$

IDA* is optimal with admissible h, uses $O(d)$ memory, but re-expands nodes across iterations — ideal when memory is scarce and h is strong. In practice, IDA* shines on puzzles (15-puzzle) where b is small.

Algorithm	$f(n)$	Optimal?	Memory
Greedy best-first	$h(n)$	No	$O(b^m)$
A* (consistent h)	$g + h$	Yes	$O(b^d)$
IDA* (admissible h)	$g + h$ threshold	Yes	$O(d)$

Table 3.1: Informed search at a glance (b branching, d depth, m max depth).

Remark 3.1 (Why heuristics matter). A better h can reduce expansions from exponential to linear. On the 8-puzzle, misplaced-tiles expands ∼10× more nodes than Manhattan at depth 20; on the 15-puzzle the gap is $> 100\times$. The cost of computing h must stay small — an $O(1)$ Manhattan lookup beats a perfect h^* that costs exponential time.

3.6 Implementing A*

```python
import heapq

def astar(start, is_goal, successors, heuristic):
    """A* graph search. successors(s) -> [(s', cost)]. Returns (path, cost)."""
    open_heap = [(heuristic(start), 0, start, [start])]  # (f, g, node, path)
```

```
    closed = {}  # node -> best g seen
    while open_heap:
        f, g, n, path = heapq.heappop(open_heap)
        if n in closed and closed[n] <= g:
            continue
        closed[n] = g
        if is_goal(n):
            return path, g
        for nxt, c in successors(n):
            ng = g + c
            nf = ng + heuristic(nxt)
            if nxt not in closed or ng < closed.get(nxt, float('inf')):
                heapq.heappush(open_heap, (nf, ng, nxt, path + [nxt]))
    return None, float('inf')

# Example: 4-neighbour grid, cost 1, Manhattan heuristic
def manhattan(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])
```

```
# Heuristic design: when is a heuristic admissible?
# 8-puzzle: tiles 1..8, blank=0, goal has blank bottom-right.

def misplaced_tiles(state, goal):
    """Counts tiles out of place. Admissible: each move fixes at most 1 tile."""
    return sum(1 for a, b in zip(state, goal) if a != 0 and a != b)

def manhattan_8puzzle(state, goal_pos):
    """Sum of Manhattan distances per tile. Dominates misplaced; still admissible."""
    dist = 0
    for idx, tile in enumerate(state):
        if tile == 0:
            continue
        r, c = divmod(idx, 3)
        gr, gc = goal_pos[tile]
        dist += abs(r - gr) + abs(c - gc)
    return dist

# Consistency check for an edge n -> n' with cost c:
#   assert heuristic(n) <= c + heuristic(n_prime)
# Straight-line distance on a map passes this (triangle inequality).

# Admissible but inconsistent example (graph search must reopen):
#   h(A)=4, h(B)=1, edge A->B cost 1 gives 4 <= 1+1? No -> inconsistent.
```

3.7 Choosing and Combining Heuristics

A heuristic comes from a *relaxed problem*: drop constraints and solve cheaply. Straight-line distance relaxes roads to Euclidean plane; Manhattan relaxes diagonal moves; misplaced-tiles relaxes that tiles interfere. If h_1, h_2 are admissible, so is $h = \max\{h_1, h_2\}$ and h dominates both (expands no more nodes, up to ties). Pattern databases precompute exact costs for subproblems (e.g., 7 tiles of the 15-puzzle) and remain admissible

by partitioning moves. The *effective branching factor* b^* solves $N + 1 = 1 + b^* + (b^*)^2 + \cdots + (b^*)^d$ for N expanded nodes and depth d; better h drives b^* toward 1.

3.8 Key Takeaways

Greedy ($f = h$) is fast but not optimal; A* ($f = g + h$) is optimal with admissible h and never reopens with consistent h; IDA* trades time for $O(d)$ memory via f-thresholds. Heuristic quality dominates performance: a dominated heuristic can expand exponentially more nodes. Design h by relaxing the problem (e.g., ignore obstacles for straight-line distance) and prefer the dominating admissible heuristic (e.g., $\max\{h_1, h_2\}$). When memory is tight, IDA* with a strong heuristic beats A*; when h is weak, A*'s best-first order saves re-expansions.

3.9 Exercises

Exercise 3.1. Prove that consistency implies admissibility. Give a 3-node graph where h is admissible but not consistent.

Exercise 3.2. Run A* on Fig. 3.1 listing the order nodes are popped from the frontier. What would greedy best-first ($f = h$) expand first?

Exercise 3.3. Show that if h_1 and h_2 are admissible then $h(n) = \max\{h_1(n), h_2(n)\}$ is admissible and dominates both. When is max still consistent?

Exercise 3.4. For the 8-puzzle, compare misplaced-tiles vs. Manhattan distance on a state where tiles 4 and 5 are swapped. Compute both values and explain which prunes more.

Exercise 3.5. IDA* starts with threshold $= h(S)$. Trace two iterations on a line $S \to A \to G$ with costs $1, 1$ and $h(S) = 2, h(A) = 2, h(G) = 0$. What is the second threshold? Why does IDA* re-expand S?

Chapter 4

Constraint Satisfaction Problems

> *"Search with structure."* A CSP separates *what* must hold (constraints) from *how* we search, so generic propagation plus backtracking solves map-colouring, scheduling, and Sudoku with the same engine.

From zero: states → constraints. We assume only state-space search from Ch. 2–3 and build CSPs from scratch: variables, domains, and relations before any algorithm. No prior logic or graph theory beyond sets.

Learning Outcomes

After this chapter you will be able to:

(L1) formalise a problem as a CSP (V, D, C) and draw its constraint graph;

(L2) trace backtracking search with MRV, degree, and LCV heuristics;

(L3) apply forward checking to prune domains and detect early failure;

(L4) define arc consistency and execute AC-3 step by step;

(L5) compare backtracking, FC, and MAC (backtracking+AC-3) trade-offs.

4.1 CSP Formalism and Constraint Graphs

Definition 4.1 (CSP)**.** A *constraint satisfaction problem* is a triple (V, D, C) where $V = \{X_1, \ldots, X_n\}$ are variables, each X_i has domain D_i (finite), and C is a set of constraints. Each $c \in C$ has scope $\text{scope}(c) \subseteq V$ and relation $\text{rel}(c) \subseteq \prod_{X_i \in \text{scope}(c)} D_i$ of allowed tuples. A *solution* assigns $x_i \in D_i$ to every X_i satisfying all c.

Unary constraints involve one variable ($X \neq \text{red}$), binary involve two ($X \neq Y$), global involve many (AllDifferent). Binary CSPs suffice in theory (auxiliary variables reduce higher arities) and dominate practice.

Example 4.1 (Australia map-colouring)**.** Variables $V = \{\mathrm{WA}, \mathrm{NT}, \mathrm{SA}, \mathrm{Q}, \mathrm{NSW}, \mathrm{V}, \mathrm{T}\}$, $D_i = \{r, g, b\}$, constraints $X \neq Y$ for adjacent regions. SA touches five neighbours, so it is most constrained—a fact heuristics will exploit.

The *constraint graph* has a node per variable and an edge per binary constraint. It reveals structure: sparse graphs are easy to cut, dense or highly connected variables bottleneck search.

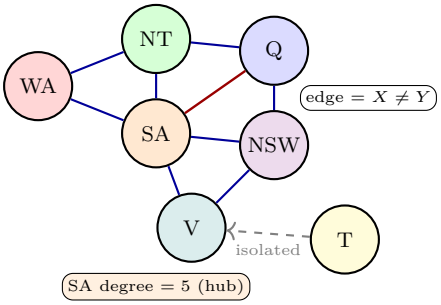

Figure 4.1: Constraint graph for Australia (binary $\neq$). Colour fills distinguish variables; SA is the high-degree hub. T is isolated (no adjacency) and can be coloured last.

4.2 Backtracking Search and Forward Checking

Naive generate-and-test enumerates $|D|^n$ assignments. Backtracking builds assignments incrementally and *prunes* when a constraint is violated: pick an unassigned X, try values in order, recurse, and backtrack on failure. Depth $\leq n$, branching $\leq d$.

Heuristics greatly cut search:

- **MRV** (minimum remaining values): pick X with smallest $|D_X|$ — fail fast.
- **Degree**: tie-break by most constraints on unassigned neighbours.
- **LCV** (least constraining value): try values that rule out fewest neighbour values first.

Forward checking (FC) maintains pruned domains: after $X \leftarrow v$, delete from each unassigned neighbour Y any value inconsistent with $X = v$. If any $D_Y = \emptyset$, backtrack immediately without deeper recursion. Stronger than plain backtracking but cheaper than full arc consistency.

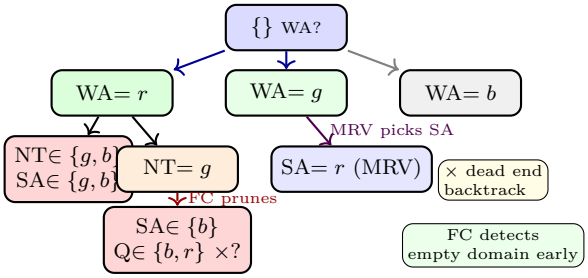

Figure 4.2: Backtracking tree with forward checking on Australia (simplified). MRV selects the most constrained variable; FC prunes neighbour domains after each assignment and cuts branches where a domain empties, before deeper recursion.

Listing 4.1: Backtracking with forward checking and MRV

```python
def backtrack(assign, domains, csp):
    if len(assign) == len(csp.vars):          # all assigned
        return assign
    var = mrv(assign, domains, csp)           # MRV + degree tie-break
    for val in lcv_order(var, domains, csp):  # least constraining first
        if consistent(var, val, assign, csp):
            new_assign = assign | {var: val}
            new_domains = forward_check(var, val, domains, csp)
            if all(new_domains[v] for v in csp.vars if v not in new_assign):
                result = backtrack(new_assign, new_domains, csp)
                if result is not None:
                    return result
    return None  # backtrack

def forward_check(var, val, domains, csp):
    nd = {v: set(d) for v, d in domains.items()}
    for nb in csp.neighbors[var]:
        if nb not in nd or val not in nd[nb]:
            nd[nb].discard(val)  # binary != constraint
    return nd
```

Forward checking is $O(nd)$ per node and already solves many sparse CSPs. When constraints are tighter (e.g. Sudoku), we need stronger propagation.

4.3 Arc Consistency and AC-3

Definition 4.2 (Arc consistency)**.** Directed arc $X \to Y$ is *arc-consistent* if for every $x \in D_X$ there exists $y \in D_Y$ with (x, y) satisfying the constraint on (X, Y). A CSP is arc-consistent if all arcs are.

Enforcing arc consistency can prune without search: if $D_{\text{SA}} = \{r\}$ and $D_{\text{WA}} = \{r, g\}$ with $SA \neq WA$, then r has no support in D_{SA} for WA, so delete r from D_{WA}.

AC-3 (Mackworth, 1977) maintains a queue of arcs. Pop $X \to Y$; if `revise`(X, Y) deletes a value, enqueue all arcs $Z \to X$ (Z neighbour of X) because X's shrinkage may break their support. Terminates when queue empty; result is the maximal arc-consistent subproblem. Complexity $O(ed^3)$ (e arcs, d domain size), $O(ed^2)$ with careful revise.

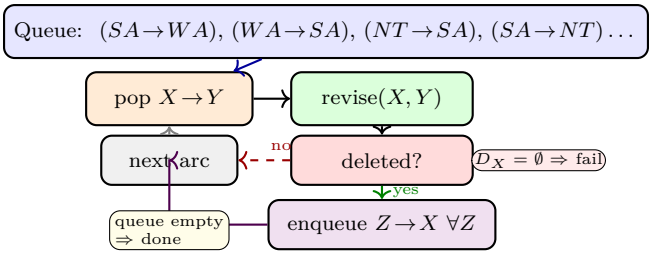

Figure 4.3: AC-3 queue processing. Each revise may shrink D_X; neighbours are re-enqueued only on deletion. Coloured stages show queue → revise → enqueue cycle until fixpoint.

Listing 4.2: AC-3 arc consistency

```python
def ac3(domains, csp):
    from collections import deque
    queue = deque((x, y) for x in csp.vars for y in csp.neighbors[x])
    while queue:
        xi, xj = queue.popleft()
        if revise(xi, xj, domains, csp):
            if not domains[xi]:          # empty domain -> unsatisfiable
                return False
            for xk in csp.neighbors[xi]:
                if xk != xj:
                    queue.append((xk, xi))
    return True

def revise(xi, xj, domains, csp):
    revised = False
    for x in list(domains[xi]):
        if not any(csp.satisfies(xi, x, xj, y) for y in domains[xj]):
            domains[xi].remove(x)
            revised = True
    return revised
```

`MAC` (maintaining arc consistency) runs AC-3 after each assignment inside backtracking. Cost per node is higher, but pruning is much stronger—often the best general CSP solver. For tree-structured graphs, AC-3 alone solves without search in $O(nd^2)$ by ordering along the tree; for general graphs search remains exponential worst-case (CSP is NP-complete).

Example 4.2 (AC-3 on a chain)**.** Variables $X, Y, Z \in \{1, 2\}$ with $X < Y$, $Y < Z$. Initially all arcs enqueued. Revise $Y \to Z$ deletes 2 from D_Y (no support > 2), leaving $D_Y = \{1\}$; this enqueues $X \to Y$. Revise $X \to Y$ deletes 1 from D_X? No—1 has support $y = 1$? Actually need $x < y$, so 1 supported by y? Wait $D_Y = \{1\}$ after pruning? Check: $Y < Z$ with $Z \in \{1, 2\}$ gives $Y \in \{1\}$ only. Then $X < Y$ with $Y = \{1\}$ leaves $D_X = \emptyset$—AC-3 proves unsatisfiability without search. FC after $X = 1$ would miss this chain effect.

Remark 4.1 (Choosing the engine)*.* Sparse map-colouring: FC+MRV often suffices. Dense Sudoku or scheduling with AllDifferent: use MAC or a dedicated global propagator (bipartite matching for AllDifferent prunes far more than pairwise $\neq$). Theory guides the trade-off: stronger propagation costs more per node but visits exponentially fewer nodes.

4.4 Chapter Notes

Further reading: Russell & Norvig Ch. 6; Dechter, *Constraint Processing*; Mackworth (1977) on AC-3 and Bessiere on AC-2001. Modern solvers add backjumping, clause learning (for SAT), and global propagators (AllDifferent via matching). Choose FC for sparse, MAC for dense, and exploit structure (cutset, tree decomposition) when the graph is nearly a tree.

4.5 Exercises

E4.1 **Warm-up: formalise.** Model 4-queens as a CSP: give V, D_i, and all binary constraints. Draw its constraint graph. How many solutions up to symmetry?

E4.2 **Heuristics trace.** On Australia with $D_i = \{r, g, b\}$, assign WA$= r$. Apply forward checking, then pick the MRV variable. List pruned domains and explain why degree tie-break favours SA.

E4.3 **Arc consistency.** Let $X, Y \in \{1, 2, 3\}$ with $X < Y$. Show the CSP is not arc-consistent, run AC-3 (queue order $X \to Y$, $Y \to X$), give each revise step and final domains. Is the result satisfiable?

E4.4 **AC-3 vs. forward checking.** Give a 3-variable CSP where FC after $X = a$ leaves domains non-empty but AC-3 detects unsatisfiability. Explain why FC misses it and what extra arc AC-3 revises.

E4.5 **MAC complexity.** Prove AC-3 is $O(ed^3)$ and $O(ed^2)$ with an optimized revise that tracks supports. On a tree with n variables, explain why a single AC-3 pass from leaves inward solves the CSP without backtracking.

Chapter 5

Adversarial Search

"Every move is a question; every reply an answer." Games pit a player against an adversary who chooses the worst reply — so optimal play means maximising under worst-case opposition, not averaging. This chapter builds that idea from zero: game trees, minimax, pruning, chance, and judging a position when time runs out.

From zero: states → opponents. We assume only state-space search from Ch. 2–3 and add one adversary who minimises your payoff. No prior game theory needed — wins, losses, and draws become numbers before any algorithm.

Learning Outcomes

After this chapter you will be able to:

(L1) define a zero-sum game $(S, A, \text{Result}, \text{Terminal}, U)$ and its game tree;

(L2) compute minimax values bottom-up and extract the optimal strategy;

(L3) trace α–β pruning and explain why it returns the same move as minimax;

(L4) evaluate chance nodes with expectiminimax for dice/card games;

(L5) design evaluation functions and cut off search sensibly under time limits.

5.1 Games and Minimax

Two-player, turn-taking, deterministic, zero-sum games (chess, tic-tac-toe, checkers) are defined by a set of states S, actions $A(s)$, transition $\text{Result}(s, a)$, terminal test $\text{Terminal}(s)$, and utility $U(s)$ from MAX's viewpoint (MIN receives $-U$). The *game tree* alternates MAX and MIN layers; leaves carry utilities.

Definition 5.1 (Minimax value)**.**

$$\text{Minimax}(s) = \begin{cases} U(s) & \text{if Terminal}(s), \\ \max_a \text{Minimax}(\text{Result}(s,a)) & \text{if Player}(s) = \text{MAX}, \\ \min_a \text{Minimax}(\text{Result}(s,a)) & \text{if Player}(s) = \text{MIN}. \end{cases}$$

MAX maximises the worst case; MIN minimises. A player's *optimal strategy* chooses, at each of their nodes, an action attaining the node's value.

Optimal play is depth-first minimax: values bubble up from the leaves. Time is $O(b^m)$ and space $O(bm)$ for branching factor b and depth m — too large for chess ($b \approx 35$, $m \approx 80$) without pruning.

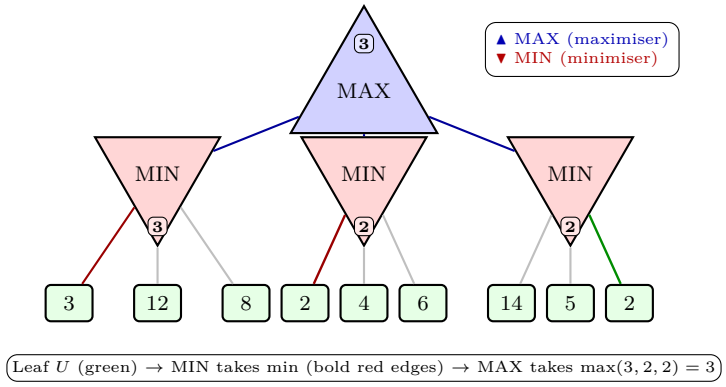

Figure 5.1: Minimax game tree. Blue up-triangles are MAX nodes, red down-triangles MIN nodes; leaf values are utilities for MAX. MIN sub-trees evaluate to $3, 2, 2$ (bold chosen edges), so the root value is $\max(3, 2, 2) = 3$.

5.2 Alpha–Beta Pruning

Minimax visits all b^m leaves. *Alpha–beta* carries two bounds down the path: α = the highest value MAX can already guarantee, β = the lowest value MIN can already guarantee. Initially $\alpha = -\infty$, $\beta = +\infty$. At a MIN node, once a child returns $\leq \alpha$, MAX would never allow this branch — it is *pruned*. Symmetrically, a MAX child $\geq \beta$ is pruned. The window $[\alpha, \beta]$ tightens as search proceeds.

Theorem 5.1 (Alpha–beta correctness)**.** With the same move ordering, alpha–beta returns the identical root value and move as minimax, visiting $O(b^{m/2})$ leaves under optimal ordering ($\approx 2b^{m/2} - 1$), and no more than b^m in the worst case.

Move ordering decides how much is pruned: ordering the best moves first (captures, killer moves) makes the bounds tighten early. Memory stays $O(bm)$ because the search is depth-first.

Listing 5.1: Minimax with alpha–beta pruning

```
def alphabeta(state, depth, alpha, beta, maximizing, game):
    """Return (value, best_move). game: actions, result, terminal, evaluate."""
    if depth == 0 or game.is_terminal(state):
        return game.evaluate(state), None
    best_move = None
    if maximizing:
```

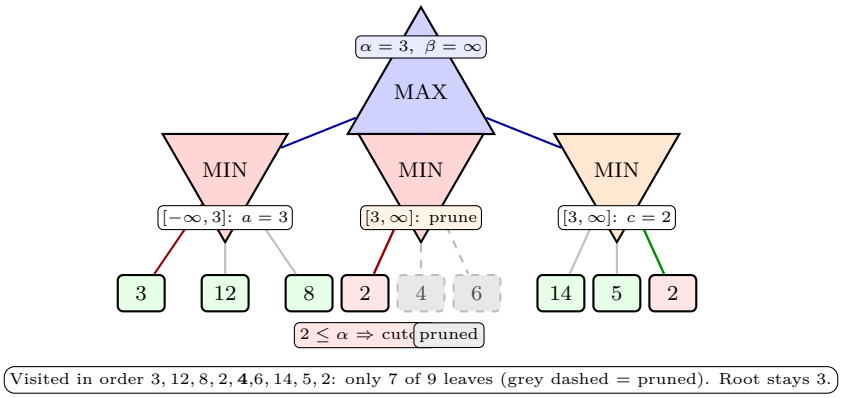

Figure 5.2: Alpha–beta on the tree of Fig. 5.1. After a returns 3, the root window is $[\alpha, \beta] = [3, \infty]$. MIN node b: the first leaf $2 \le \alpha$ already caps b at 2, so the remaining leaves 4, 6 are pruned (grey dashed). MIN node c is fully scanned $(14, 5, 2)$ but can only reach 2, so the root value 3 is unchanged — identical to full minimax.

```
        value = float('-inf')
        for move in game.actions(state):      # order best-first for pruning!
            child = game.result(state, move)
            score, _ = alphabeta(child, depth-1, alpha, beta, False, game)
            if score > value:
                value, best_move = score, move
            alpha = max(alpha, value)
            if value >= beta:                 # beta cutoff
                break
        return value, best_move
    else:
        value = float('inf')
        for move in game.actions(state):
            child = game.result(state, move)
            score, _ = alphabeta(child, depth-1, alpha, beta, True, game)
            if score < value:
                value, best_move = score, move
            beta = min(beta, value)
            if value <= alpha:                # alpha cutoff
                break
        return value, best_move

# Usage: val, move = alphabeta(root, 4, -float('inf'), float('inf'), True, g)
```

Minimax without pruning would be the same code without the two cutoff lines; the cutoffs only remove branches that cannot change the result, which is why Theorem 5.1 holds. Do not forget the comment: *move ordering is what makes alpha–beta fast*; with random ordering the gain shrinks toward none in the worst case.

5.3 Chance Nodes and Expectiminimax

Backgammon, Monopoly, and card games add randomness. We insert *chance* nodes whose children are the dice or card outcomes d_i, each with known probability $P(d_i)$; the node's value is an expectation:

$$\text{Expectiminimax}(s) = \begin{cases} U(s) & \text{terminal}, \\ \max_a \text{Expectiminimax}(\text{Result}(s,a)) & \text{MAX}, \\ \min_a \text{Expectiminimax}(\text{Result}(s,a)) & \text{MIN}, \\ \sum_i P(d_i)\,\text{Expectiminimax}(\text{Result}(s,d_i)) & \text{CHANCE}. \end{cases}$$

Time becomes $O(b^m n^m)$ when each chance node has n outcomes, and pruning weakens: a bound on *expectations* is rarely decisive. Still, bounded-window variants (Ballard's STAR algorithms: probe children of a chance node until the expectation exits $[\alpha, \beta]$) prune safely and are used in practice.

Listing 5.2: Expectiminimax with chance nodes

```
def expectiminimax(state, depth, game):
    """Expectiminimax. Chance nodes use game.chance_outcomes(state)
    -> list of (outcome, probability)."""
    if depth == 0 or game.is_terminal(state):
        return game.evaluate(state), None
    player = game.player(state)
    if player == 'CHANCE':
        exp_value = 0.0
        for outcome, prob in game.chance_outcomes(state):
            child = game.chance_result(state, outcome)
            val, _ = expectiminimax(child, depth - 1, game)
            exp_value += prob * val
        return exp_value, None
    elif player == 'MAX':
        best, best_move = float('-inf'), None
        for move in game.actions(state):
            child = game.result(state, move)
            val, _ = expectiminimax(child, depth - 1, game)
            if val > best:
                best, best_move = val, move
        return best, best_move
    else:                                   # MIN
        best, best_move = float('inf'), None
        for move in game.actions(state):
            child = game.result(state, move)
            val, _ = expectiminimax(child, depth - 1, game)
            if val < best:
                best, best_move = val, move
        return best, best_move

# Backgammon: chance_outcomes = 36 dice pairs (i,j), each with prob 1/36
```

Caveat: expectiminimax optimises the expected utility. A risk-averse player may prefer a guaranteed 4 over an expected 5 with variance (utility theory, not search, decides); the algorithm itself assumes utilities already encode preferences.

5.4 Evaluation Functions and Cutoff

Full search to terminal nodes is impossible except for tiny games. Instead we *cut off* at depth d and apply an *evaluation function* $\text{Eval}(s) \approx U(s)$. For chess, material counts ($P = 1,\ N = B = 3,\ R = 5,\ Q = 9$) plus mobility and pawn structure are classic; modern engines use neural networks trained by self-play.

A good Eval is fast, monotone (wins > 0, losses < 0, draws 0), and strongly correlated with the chance of winning. Two pitfalls matter:

- **Horizon effect**: a forced win or loss just beyond the cutoff looks fine; quiescence search extends the frontier through captures and checks until the position is *quiescent*.
- **Stability**: material counts swing wildly between adjacent plies; demanding a symmetric score, $\text{Eval}(s) = -\text{Eval}(s')$ for mirrored s, s', halves the variance.

Engineering the full player: *iterative deepening* (search 1,2,3,... plies until the clock runs out), alpha–beta with strong move ordering (captures first, killer heuristic), a *transposition table* (hash visited states to reuse values), and quiescence search at the horizon.

Remark 5.1 (Why this chapter matters in practice). Every strong chess/draughts/shogi program since the 1970s is exactly this pipeline; AlphaZero replaced the hand-written Eval with a learned one but kept the search tree. Understanding minimax and α–β is understanding the engine's skeleton.

5.5 Chapter Notes

Further reading: Russell & Norvig Ch. 5; Knuth & Moore (1975) on alpha–beta completeness; Ballard (1983) on expectation pruning; Shannon (1950) on evaluation functions for chess. Exercises below build tic-tac-toe and a coin-flip game by hand first, then by code.

5.6 Exercises

E5.1 **Minimax by hand.** For Fig. 5.1, compute the value of every internal node bottom-up. Which root move is optimal and what is the minimax value? If MIN plays suboptimally exactly once (say 12 from node a), how much better can MAX do?

E5.2 **Alpha–beta trace.** Replay Fig. 5.2 left to right, updating $[\alpha, \beta]$ after every leaf. List the exact order of leaf visits and the pruned leaves. How many leaves does full minimax visit vs. alpha–beta? Would pruning change if the root's children were ordered c, b, a?

E5.3 **Move ordering.** Reorder each MIN node's leaves in Fig. 5.1 to (a) maximise pruning and (b) minimise it. Count visited leaves in both cases and justify the $O(b^{m/2})$ bound of Theorem 5.1.

E5.4 **Expectiminimax.** MAX picks A or B; then a fair coin lands H or T. Payoffs (MAX's utility): $A{+}H = 5$, $A{+}T = 1$, $B{+}H = 3$, $B{+}T = 4$. Draw the tree (with CHANCE nodes), compute expectiminimax values, and give MAX's optimal choice. How does a biased coin with $P(H) = 0.7$ change it? Why can alpha–beta

not prune the losing chance branch here?

E5.5 **Evaluation design.** Design Eval for tic-tac-toe as the number of open lines (row/column/diagonal) free of MIN marks and containing at least one MAX mark, minus the same count for MIN. Compute it for (a) X in the centre, (b) X in a corner, on an empty board, and state which opening it prefers. Explain the horizon effect: with cutoff at depth 1, a position where MAX can fork in two moves can score worse than a calm, materially better one.

Chapter 6

Logic and Knowledge Representation

> *"Logic is the calculus of reasoning."* A knowledge-based agent separates *what it knows* (a knowledge base) from *how it reasons* (inference). Logic gives both a language and a guarantee: if the premises entail a sentence, a complete proof procedure will find it.

From zero: search → knowledge. Ch. 2–4 searched state spaces; this chapter asks how an agent *represents* and *derives* knowledge. We build propositional logic from truth tables, then lift it to first-order logic. No prior logic assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) formalise sentences in propositional and first-order logic (FOL);

(L2) test entailment $KB \models \alpha$ via truth tables and convert to CNF;

(L3) apply resolution, forward chaining and backward chaining;

(L4) interpret FOL semantics (domains, interpretations, models) and compute unifiers;

(L5) trace FOL resolution with unification and standardising apart.

6.1 Propositional Logic: Syntax, Semantics, Entailment

A *proposition* is a statement that is true or false. Propositional atoms P, Q, R combine with connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$. A *model* m assigns each atom T/F; semantics is the usual truth tables ($\neg$, $\wedge$, $\vee$, $\Rightarrow \equiv \neg P \vee Q$).

$KB \models \alpha$ (entailment) iff every model of KB satisfies α. Equivalently, $KB \wedge \neg\alpha$ is unsatisfiable — the basis for refutation.

CNF (conjunctive normal form) is a conjunction of clauses, each a disjunction of literals. Every sentence

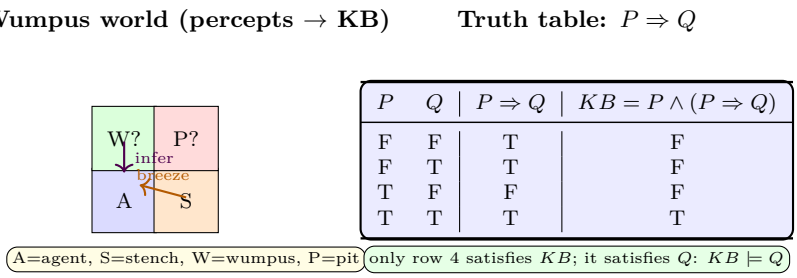

Figure 6.1: Left: Wumpus grid — percepts (stench, breeze) enter KB as sentences. Right: truth table shows entailment $\{P, P \Rightarrow Q\} \models Q$; only the KB-models matter.

converts via: eliminate $\Leftrightarrow, \Rightarrow$, push $\neg$ inward (De Morgan), distribute $\vee$ over $\wedge$.

6.2 Inference: Resolution, Forward and Backward Chaining

The *resolution rule* on clauses $(\ell \vee C_1)$ and $(\neg\ell \vee C_2)$ derives $(C_1 \vee C_2)$ (the resolvent). Resolution is sound and *refutation-complete* for CNF: to prove $KB \models \alpha$, add $\neg\alpha$ to KB, convert to CNF, and derive the empty clause $\square$.

Forward chaining (FC) fires *definite clauses* $(p_1 \wedge \cdots \wedge p_k \Rightarrow q)$ whenever premises are known, adding q until fixpoint — data-driven, $O(n)$ for Horn KBs, complete for Horn. Backward chaining (BC) is goal-driven: to prove q, prove its premises recursively (depth-first, as in Prolog), with memoisation.

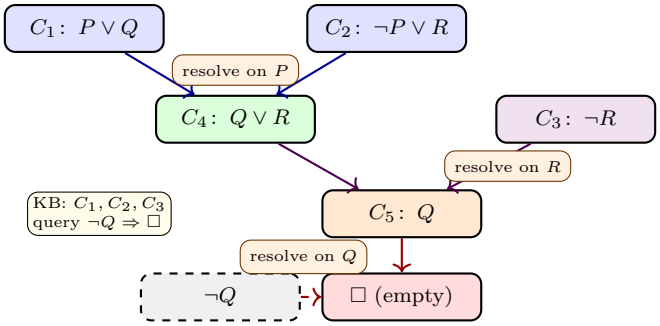

Figure 6.2: Resolution proof tree (refutation). Adding $\neg Q$ to $KB = \{P \vee Q,\ \neg P \vee R,\ \neg R\}$ yields $\square$; hence $KB \models Q$. Coloured levels show successive resolvents.

Listing 6.1: Propositional resolution (refutation) on CNF clauses

```
def resolve(ci, cj):
    """ci, cj are frozensets of literals like 'P' or '~P'. Yield resolvents."""
    for lit in ci:
        comp = lit[1:] if lit.startswith("~") else "~"+lit
        if comp in cj:
            resolvent = (ci - {lit}) | (cj - {comp})
            # tautology check: contains both x and ~x
            if not any((l[1:] if l.startswith("~") else "~"+l) in resolvent for l in
                resolvent):
                yield frozenset(resolvent)

def pl_resolution(kb, query):
    """kb: list of clauses; query: literal. Returns True if kb |= query."""
    neg = "~" + query if not query.startswith("~") else query[1:]
```

```
    clauses = set(kb) | {frozenset([neg])}   # add negated query
    new = set()
    while True:
        for ci in list(clauses):
            for cj in list(clauses):
                if ci is cj: continue
                for r in resolve(ci, cj):
                    if len(r) == 0: return True   # empty clause
                    new.add(r)
        if new.issubset(clauses): return False
        clauses |= new

# KB = {P v Q, ~P v R, ~R}  entails Q ?
kb = [frozenset({"P","Q"}), frozenset({"~P","R"}), frozenset({"~R"})]
print(pl_resolution(kb, "Q"))  # True
```

6.3 First-Order Logic: Syntax, Semantics, Unification

Propositional logic cannot say "every wumpus is smelly" without enumerating. FOL adds *terms* (constants, variables, functions), *predicates*, *quantifiers* $\forall, \exists$, and equality.

Definition 6.1 (FOL sentence)**.** Terms $t ::= c \mid x \mid f(t_1, \dots, t_n)$; atoms $P(t_1, \dots, t_n)$; sentences $\varphi ::= \text{atom} \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \Rightarrow \varphi \mid \forall x\, \varphi \mid \exists x\, \varphi$.

Semantics: an *interpretation* $\mathcal{I} = (D, \cdot^{\mathcal{I}})$ has domain $D \neq \emptyset$ mapping each constant to D, function to $D^n \to D$, predicate to a relation on D. $\forall x\, \varphi$ holds iff φ holds for every $d \in D$ (with $x \mapsto d$).

Unification finds a substitution $\theta = \{x_1/t_1, \dots\}$ making two literals identical: $\textsc{Unify}(P(x, f(a)), P(b, y))$ needs $\{x/b,\ y/f(a)\}$. The *most general unifier* (MGU) is unique up to renaming; occur-check (x in t) prevents unsound cycles.

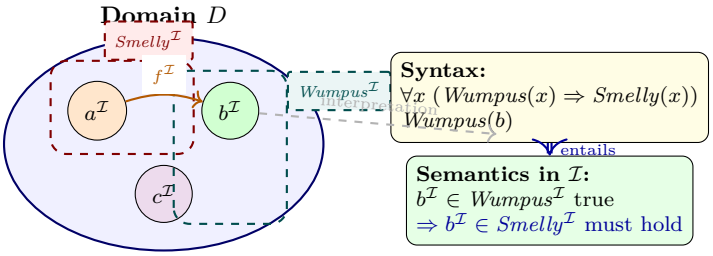

Figure 6.3: FOL semantics: syntax maps into a domain D via $\mathcal{I}$. Predicates are relations (dashed regions); functions map elements. The sentence $\forall x(W(x) \Rightarrow S(x))$ constrains every element of D.

FOL resolution lifts propositional resolution with unification. Clauses are universally quantified disjunctions; variables are *standardised apart* (renamed) before resolving. From $\{\neg P(x) \vee Q(x),\ P(a)\}$ with $\theta = \{x/a\}$ derive $Q(a)$.

Listing 6.2: FOL unification (MGU with occur-check)

```
def unify(x, y, theta=None):
    """Robinson unification. Terms are str vars ('?x') or [fun, args...]. Returns dict or
       None."""
```

```
    if theta is None: theta = {}
    if theta is None: return None
    if x == y: return theta
    if isinstance(x, str) and x.startswith("?"): return unify_var(x, y, theta)
    if isinstance(y, str) and y.startswith("?"): return unify_var(y, x, theta)
    if isinstance(x, list) and isinstance(y, list):
        if x[0] != y[0] or len(x) != len(y): return None
        for xi, yi in zip(x[1:], y[1:]):
            theta = unify(xi, yi, theta)
            if theta is None: return None
        return theta
    return None

def unify_var(var, x, theta):
    if var in theta: return unify(theta[var], x, theta)
    if isinstance(x, str) and x in theta: return unify(var, theta[x], theta)
    if occurs(var, x, theta): return None  # occur-check
    theta = dict(theta); theta[var] = x; return theta

def occurs(var, x, theta):
    if x == var: return True
    if isinstance(x, list): return any(occurs(var, a, theta) for a in x[1:])
    if isinstance(x, str) and x in theta: return occurs(var, theta[x], theta)
    return False

# e.g. Knows(?x, f(?y))  ~  Knows(John, f(Mary))
print(unify(["Knows","?x",["f","?y"]], ["Knows","John",["f","Mary"]]))
# {'?x': 'John', '?y': 'Mary'}
print(unify("?x", ["f","?x"]))  # None (occur-check)
```

FOL resolution strategy: convert $KB \wedge \neg\alpha$ to CNF (Skolemise $\exists$ to fresh functions), then repeatedly resolve complementary literals with MGU, standardising apart, until $\square$ or saturation. It is refutation-complete for FOL; general entailment is only semi-decidable.

6.4 Key Takeaways

Propositional inference reduces to CNF + resolution ($\square$ means entailment); Horn KBs also allow linear forward/backward chaining. FOL adds expressive power via quantifiers, at the cost of undecidability — unification + lifted resolution is the algorithmic bridge, complete for refutation but only semi-decidable in general. Design KB for the task: propositional for small worlds, FOL when relations quantify over many objects.

6.5 Exercises

E6.1 **Entailment by truth table.** Show $\{P \Rightarrow Q,\ Q \Rightarrow R\} \models P \Rightarrow R$ using Fig. 6.1 style. How many rows? Which rows are KB-models?

E6.2 **CNF + resolution.** Convert $(P \Rightarrow Q) \wedge (Q \Rightarrow R) \wedge P$ and query R to CNF, then give a resolution tree

(as in Fig. 6.2) deriving $\square$ from $KB \cup \{\neg R\}$.

E6.3 **Chaining.** Horn KB: $P,\ P \Rightarrow Q,\ Q \wedge R \Rightarrow S$. Trace forward chaining from $\{P, R\}$ and backward chaining for goal S. Which explores fewer clauses and why?

E6.4 **Unification.** Find the MGU (or say why none exists) for each pair:

- $P(?x, f(?x))$ vs. $P(a, f(b))$;
- $P(?x, ?y)$ vs. $P(f(?y), b)$;
- $Knows(?x, ?x)$ vs. $Knows(John, Mary)$,

and explain any occur-check failure above.

E6.5 **FOL resolution.** From $\forall x(W(x) \Rightarrow S(x))$ and $W(b)$, prove $S(b)$ by FOL resolution: Skolemise, clausify, standardise apart, unify, and show the tree ending in $\square$ (use Fig. 6.3 interpretation to argue soundness).

Chapter 7

Automated Planning

"Planning is reasoning about doing." We leave behind explicit graph search on hand-coded states and instead let the agent *describe* the world in logic — then *derive* a sequence of actions that achieves a goal.

From zero: search → descriptions. We assume only state-space search (Ch. 2–3) and propositional logic (Ch. 6). We build planning from scratch: what a state *is*, how an action *changes* it, then how to plan efficiently. No prior PDDL or SAT assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) formalise a planning problem with STRIPS operators (preconditions, add, delete);

(L2) contrast state-space and plan-space search;

(L3) build a planning graph and identify mutex relations;

(L4) trace Graphplan (expand → extract) on a small example;

(L5) encode bounded planning as SAT and sketch PDDL syntax.

7.1 The Planning Problem and STRIPS

A *planning problem* is a triple (S_0, G, O): initial state S_0 (a set of true propositions), goal G (a conjunction to achieve), and operators O. The *closed-world assumption* holds: anything not listed is false.

Definition 7.1 (STRIPS operator)**.** An operator o is $\langle \text{Name}(o), \text{Pre}(o), \text{Add}(o), \text{Del}(o) \rangle$ where $\text{Pre}(o)$ are preconditions that must hold, $\text{Add}(o)$ are propositions made true, and $\text{Del}(o)$ are propositions made false. o is *applicable* in S iff $\text{Pre}(o) \subseteq S$; the result is $S' = (S \setminus \text{Del}(o)) \cup \text{Add}(o)$.

Example 7.1 (Blocks world)**.** Move(b, x, y): move block b from x to y. Pre: $\{\text{On}(b, x), \text{Clear}(b), \text{Clear}(y)\}$, Add: $\{\text{On}(b, y), \text{Clear}(x)\}$, Del: $\{\text{On}(b, x), \text{Clear}(y)\}$. Grounding b, x, y over blocks yields propositional operators.

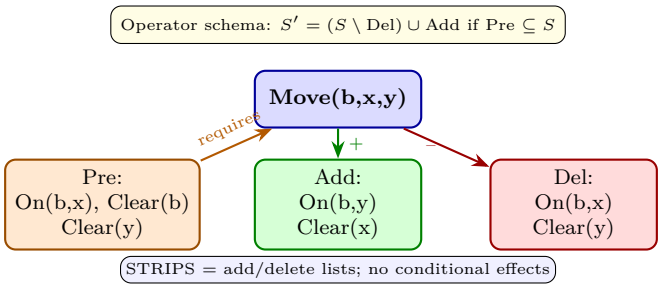

Figure 7.1: STRIPS operator anatomy. Preconditions gate applicability (orange); effects split into add list (green, propositions asserted) and delete list (red, propositions retracted).

State-space size is $2^{|\mathcal{P}|}$ for $|\mathcal{P}|$ propositions — far too large to enumerate. Planning exploits the *factored* representation to search more cleverly.

7.2 State-Space vs. Plan-Space Search

State-space planners search over world states: forward (progression) from S_0 applying applicable operators, or backward (regression) from G applying operators relevant to open goals. Forward branching is $|O|$; backward branching is limited to operators that achieve a goal literal but may introduce many subgoals.

Plan-space planners search over *partial plans* (sets of steps with ordering and causal-link constraints) and refine by resolving flaws (open precondition or threat). Plan-space enables least-commitment: order a before b only when needed. Modern planners mostly use state-space with heuristics (e.g., planning-graph heuristics), but plan-space clarifies the structure of reasoning.

	State-space	Plan-space
Node	world state S	partial plan $(A, \prec, \text{CL})$
Start	S_0	empty plan + start/finish steps
Successor	apply one operator	repair one flaw
Heuristic	$h(S)$ e.g. relaxed plan	fewest flaws

7.3 Planning Graphs and Mutex

A *planning graph* compactly approximates reachability in layers. Level $P_0 = S_0$ (propositions). Then alternate action layers A_i and proposition layers P_{i+1}: A_i contains every operator whose preconditions appear in P_i (plus persistence/no-op actions), and P_{i+1} contains all adds of A_i. Edges link preconditions to actions and actions to effects. Crucially, we mark *mutual exclusions* (mutex).

Two actions at the same level are mutex if (i) one deletes a precondition or add of the other (*interference*), or (ii) they have mutex preconditions (*competing needs*). Two propositions are mutex if all ways to achieve them are pairwise mutex (including persistence). Mutex propagation prunes impossible combinations without full search.

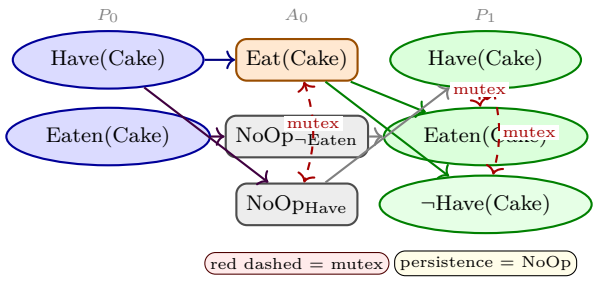

Figure 7.2: Planning graph for the cake problem (one eat step). $P_0 \to A_0 \to P_1$ with precondition/effect edges. Red dashed arcs are mutex: Eat(Cake) interferes with $\text{NoOp}_{\text{Have}}$; Have and Eaten become mutex at P_1.

If $G \subseteq P_k$ with no mutex among goal propositions, a solution *might* exist within k steps. If the graph levels off (no new propositions or mutexes), and G remains mutex, the problem is unsolvable.

Listing 7.1: Forward BFS planner over STRIPS states

```python
from collections import deque

def strips_bfs(init, goal, operators):
    """BFS progression planner. State = frozenset of true propositions."""
    frontier = deque([(frozenset(init), [])])
    visited = {frozenset(init)}
    while frontier:
        state, plan = frontier.popleft()
        if goal.issubset(state):
            return plan
        for op in operators:
            if op['pre'].issubset(state):          # applicable?
                nxt = (state - op['del']) | op['add']
                if nxt not in visited:
                    visited.add(nxt)
                    frontier.append((nxt, plan + [op['name']]))
    return None  # no plan

# Example: cake problem
ops = [
    {'name':'Eat(Cake)', 'pre':{'Have(Cake)'}, 'add':{'Eaten(Cake)'}, 'del':{'Have(Cake)'}},
    {'name':'Bake(Cake)', 'pre':{'NotHave(Cake)'}, 'add':{'Have(Cake)'}, 'del':{'NotHave(Cake)'}},
]
print(strips_bfs({'Have(Cake)'}, {'Have(Cake)','Eaten(Cake)'}, ops))
```

7.4 Graphplan

Graphplan (Blum & Furst, 1997) interleaves expansion and extraction:

1. **Expand** the planning graph one level until G appears non-mutex in P_k.
2. **Extract** a plan by backward search: pick actions in A_{k-1} achieving G, ensure they are pairwise non-mutex, recurse on their preconditions at P_{k-1}. On failure, backtrack; if no set works, expand further.

3. If the graph levels off with G still mutex, report unsolvable.

Graphplan is sound, complete, and often much faster than naive BFS because mutex pruning eliminates vast dead space. The extracted plan is parallel: non-mutex actions at the same level run concurrently.

7.5 SAT Planning and PDDL

Bounded planning asks: is there a plan of length $\le k$? Encode as SAT and let a solver decide. Create variables p_t (proposition p at time t) and a_t (action a at time t) for $0 \le t \le k$:

$$\text{initial: } p_0 \text{ for } p \in S_0,\ \neg p_0 \text{ otherwise} \quad \text{goal: } p_k \text{ for } p \in G$$

$$\text{action} \to \text{pre: } a_t \to \bigwedge_{p \in \mathrm{Pre}(a)} p_t \quad \text{action} \to \text{eff: } a_t \to \bigwedge_{p \in \mathrm{Add}(a)} p_{t+1} \wedge \bigwedge_{p \in \mathrm{Del}(a)} \neg p_{t+1}$$

$$\text{frame: } (p_t \wedge \neg p_{t+1}) \to \bigvee_{a : p \in \mathrm{Del}(a)} a_t \quad \text{exclusion: at most one } a_t \text{ per } t$$

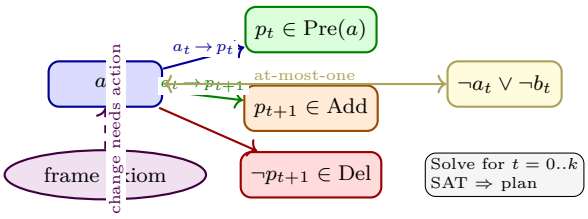

Figure 7.3: SAT encoding of bounded planning. Each action implies its preconditions and effects; frame axioms force every change to be caused by an action; exclusion clauses enforce serial plans. A satisfying assignment at horizon k is a k-step plan.

Iteratively increase k until SAT (plan found) or resource limit. Modern SAT planners (Blackbox, SATPlan) add planning-graph mutex as clauses to speed solving.

PDDL basics. The Planning Domain Definition Language describes domains and problems separately:

```
(:action Move :parameters (?b ?x ?y) :precondition (and (On ?b ?x) (Clear ?b) (Clear ?y))
          :effect (and (On ?b ?y) (Clear ?x) (not (On ?b ?x)) (not (Clear ?y))))
```

A problem file gives `:init` and `:goal`; grounding instantiates lifted operators. PDDL 2.1 adds numeric fluents and durative actions.

Listing 7.2: Mutex detection for a planning-graph level

```
def action_mutex(a, b, prop_mutex):
    """True if actions a,b are mutex (interference or competing needs)."""
    if a['add'] & b['del'] or b['add'] & a['del']:
        return True   # interference
    if a['del'] & b['pre'] or b['del'] & a['pre']:
        return True
    if a['pre'] & b['add'] or b['pre'] & a['add']:
        pass  # covered by competing needs below
    for p in a['pre']:
```

```
        for q in b['pre']:
            if (p, q) in prop_mutex or (q, p) in prop_mutex:
                return True  # competing needs
    return False

def prop_mutex_level(props, achievers, act_mutex):
    """Props p,q mutex if every achiever pair is mutex."""
    mutex = set()
    plist = list(props)
    for i, p in enumerate(plist):
        for q in plist[i+1:]:
            all_mutex = True
            for ap in achievers.get(p, []):
                for aq in achievers.get(q, []):
                    if ap is aq:
                        all_mutex = False; break
                    if (ap, aq) not in act_mutex and (aq, ap) not in act_mutex:
                        all_mutex = False; break
                if not all_mutex:
                    break
            if all_mutex and achievers.get(p) and achievers.get(q):
                mutex.add((p, q))
    return mutex
```

7.6 Key Takeaways

STRIPS factors the world into propositions and operators; state-space progression/regression search from this factored form. Planning graphs over-approximate reachability and expose mutex to prune massively. Graphplan extracts a parallel plan by backward search over the graph, expanding only as needed. Bounded planning compiles to SAT (action→pre/eff + frame + exclusion) and leverages modern solvers. PDDL cleanly separates lifted domain from grounded problem.

7.7 Chapter Notes

Further reading: Russell & Norvig Ch. 10–11; Blum & Furst (1997) on Graphplan; Kautz & Selman (1996) on SATPlan. Heuristic search planning (FF, Fast Downward) dominates today, using planning-graph relaxations (h_{add}, $h_{\max}$, h_{FF}) to guide forward search. Try the `fast-downward` or `pyperplan` toolkits with PDDL benchmarks (Blocks, Logistics, Rovers).

7.8 Exercises

E7.1 **Warm-up: STRIPS grounding.** Domain has predicates On/2, Clear/1 and blocks $\{A, B, C\}$. Write the grounded operator Move(A,B,C) with pre/add/del sets. How many grounded Move operators exist?

E7.2 **State-space vs. plan-space.** For a problem with branching $b = 6$ and solution depth $d = 5$, compare

worst-case nodes for forward BFS vs. a plan-space search that keeps 3 partial plans. When does regression beat progression?

E7.3 **Planning graph trace.** Initial $\{P, Q\}$, actions A_1: Pre $\{P\} \to$ Add $\{R\}$, A_2: Pre $\{Q\} \to$ Add $\{S\}$, A_3: Pre $\{R, S\} \to$ Add $\{G\}$ (all delete $\emptyset$). Build P_0, A_0, P_1, A_1, P_2 and mark all mutex. At which level does G first appear non-mutex with itself?

E7.4 **Graphplan extraction.** Use the graph from E7.3 to extract a plan for goal $\{G\}$. Show the backward search choices at each level and give the final parallel plan. What changes if A_1 deletes Q?

E7.5 **SAT encoding.** Encode the cake problem (Have $\to$ Eat $\to$ Eaten) for horizon $k = 1$ as CNF: list all variables p_t, a_t and write the clauses for initial, goal, action$\to$pre/eff, frame, and at-most-one. Is the formula satisfiable? What about $k = 0$?

Chapter 8

Learning from Data

> *"A program learns from experience E with respect to task T and performance P if its performance on T, as measured by P, improves with E."* — Tom Mitchell. We make this concrete: from data to hypotheses, from entropy to trees, from a single neuron to a network.

From zero: no ML assumed. We build from counting and Ch. 1 agents only: training data as labelled examples, generalisation as the goal, then two pillars—decision trees (discrete, interpretable) and the perceptron (continuous, neural preview). No prior statistics required.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish supervised, unsupervised, and reinforcement learning;

(L2) explain training/test split and overfitting;

(L3) compute entropy and information gain and trace ID3;

(L4) run k-NN and describe the bias of linear hypotheses;

(L5) execute the perceptron rule and explain why XOR needs a hidden layer.

8.1 What Is Learning?

An agent with fixed rules fails when the world changes. A *learning* agent improves its mapping $h : \mathcal{X} \to \mathcal{Y}$ from experience. Data are pairs $(x^{(i)}, y^{(i)})$ where x is a feature vector and y is the label.

Three regimes differ only in feedback:

Supervised (x, y) given; learn $h \approx f$. Classification (y discrete) or regression (y continuous).

Unsupervised Only x given; find structure (clusters, density).

Reinforcement Agent acts, receives reward r, must credit actions to rewards.

We focus on supervised learning. Fix a *hypothesis space* $\mathcal{H}$ (e.g., all depth-2 trees). The learner picks $\hat{h} \in \mathcal{H}$ minimising training error, but we care about *test error* on unseen data. A large $\mathcal{H}$ can memorise training data yet fail on new inputs—*overfitting*. Split data into train/validate/test: fit on train, choose $\mathcal{H}$ on validate, report once on test.

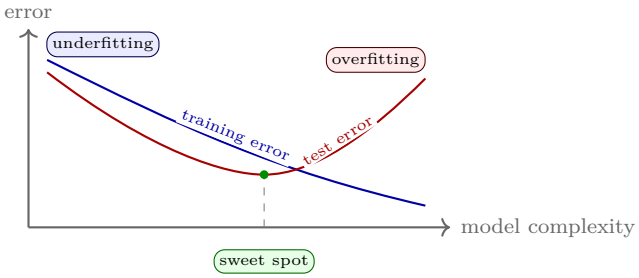

Figure 8.1: Training error falls with complexity; test error is U-shaped. The gap is overfitting—memorising noise, not signal.

8.2 Decision Trees: Entropy and Information Gain

A *decision tree* classifies by asking a sequence of feature tests. Each internal node tests an attribute, each branch is a value, each leaf is a class.

Which attribute first? The one that reduces uncertainty most. For set S with class proportions p_c,

$$H(S) = -\sum_c p_c \log_2 p_c \quad \text{(entropy, bits)},$$

with $0 \log 0 := 0$. $H = 0$ when pure; $H = 1$ for a fair coin. Splitting S on attribute A into subsets S_v gives

$$\text{Gain}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v).$$

ID3 greedily picks $\arg\max_A \text{Gain}(S, A)$ and recurses.

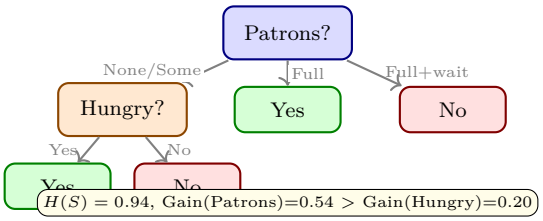

Figure 8.2: Decision tree for the restaurant domain. Root splits on Patrons because it maximises gain. Green = positive, red = negative.

Example 8.1 (Entropy calculation)**.** 12 examples: 6 Yes, 6 No gives $H(S) = 1$ bit. Splitting on Patrons yields None (0Y,2N, $H = 0$), Some (4Y,0N, $H = 0$), Full (2Y,4N, $H = 0.92$). Weighted entropy $= 0.46$, so Gain $= 0.54$ bits.

k-NN is the opposite extreme: no tree, just vote among the k nearest training points (Euclidean distance). Small k overfits; large k over-smooths. Linear regression fits $\hat{y} = w^\top x + b$ by minimising squared error via gradient descent—the same update shape the perceptron uses.

Listing 8.1: ID3 decision-tree learner (categorical features)

```python
import math
from collections import Counter

def entropy(labels):
    n = len(labels)
    return -sum((c/n)*math.log2(c/n) for c in Counter(labels).values())

def info_gain(data, attr, label_key='y'):
    base = entropy([r[label_key] for r in data])
    vals = set(r[attr] for r in data)
    weighted = sum(len([r for r in data if r[attr]==v])/len(data)
                   * entropy([r[label_key] for r in data if r[attr]==v])
                   for v in vals)
    return base - weighted

data = [
    {'Patrons':'None','Hungry':'Yes','y':'No'},
    {'Patrons':'Some','Hungry':'Yes','y':'Yes'},
    {'Patrons':'Full','Hungry':'Yes','y':'Yes'},
    {'Patrons':'Full','Hungry':'No','y':'No'},
]
print("Gain(Patrons)=%.2f" % info_gain(data, 'Patrons'))
print("Gain(Hungry)=%.2f" % info_gain(data, 'Hungry'))
```

8.3 The Perceptron and Neural Preview

A *perceptron* is one artificial neuron. With inputs x, weights w, bias b,

$$z = w \cdot x + b, \qquad \hat{y} = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

so $\hat{y} = \text{step}(z)$. Geometrically, $w \cdot x + b = 0$ is a hyperplane separating classes.

Learning rule. On mistake (x, y), $y \in \{0, 1\}$,

$$w \leftarrow w + \eta\,(y - \hat{y})\,x, \qquad b \leftarrow b + \eta\,(y - \hat{y}),$$

with rate $\eta > 0$. If data are linearly separable, this converges in finite steps (Block–Novikoff). Otherwise it cycles—XOR is the classic impossibility: no single line separates $\{(0,0),(1,1)\}$ from $\{(0,1),(1,0)\}$.

Stacking perceptrons with a smooth activation (sigmoid, ReLU) yields a *feed-forward network*. With one hidden layer it is a universal approximator; training via gradient descent (backpropagation) minimises cross-

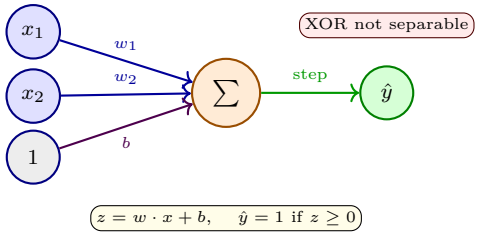

Figure 8.3: Perceptron as a linear separator. Stacking such units with non-linear activations gives a multi-layer network that solves XOR.

entropy—the bridge to deep learning.

Listing 8.2: Perceptron learning rule from scratch

```python
import numpy as np

def perceptron_train(X, y, eta=0.1, epochs=20):
    """X: (n,d) array, y in {0,1}. Returns (w,b)."""
    n, d = X.shape
    w = np.zeros(d)
    b = 0.0
    for _ in range(epochs):
        err = 0
        for xi, yi in zip(X, y):
            yhat = 1 if np.dot(w, xi) + b >= 0 else 0
            if yhat != yi:
                w += eta * (yi - yhat) * xi
                b += eta * (yi - yhat)
                err += 1
        if err == 0:
            break  # converged: linearly separable
    return w, b

X_and = np.array([[0,0],[0,1],[1,0],[1,1]])
y_and = np.array([0,0,0,1])
y_xor = np.array([0,1,1,0])
print("AND w:", perceptron_train(X_and, y_and)[0])
w, b = perceptron_train(X_and, y_xor, epochs=10)
print("XOR solved?", np.all(((X_and.dot(w)+b)>=0).astype(int)==y_xor))
```

8.4 Key Takeaways

Supervised learning picks $\hat{h} \in \mathcal{H}$ to minimise training error but is judged on test error; overfitting grows with $\mathcal{H}$ complexity. Decision trees greedily maximise information gain and are interpretable but need pruning. The perceptron learns a linear separator by mistake-driven updates and converges iff data are separable; stacking units with non-linearities yields deep networks trained by gradient descent.

8.5 Chapter Notes

Further reading: Russell & Norvig Ch. 18–19; Mitchell Ch. 3 (trees) and Ch. 4 (perceptron); Goodfellow et al. Ch. 6 for deep nets. Try scikit-learn for trees/k-NN and PyTorch for perceptrons/MLPs; always report validation curves, not just training accuracy.

8.6 Exercises

E8.1 **Paradigms.** For (a) spam filter from labelled emails, (b) customer segmentation from purchase histories, (c) game agent from win/loss rewards, state the regime (supervised/unsupervised/RL) and what (x, y) or reward is.

E8.2 **Entropy and gain.** Dataset S: 9 Yes, 5 No. Attribute A splits into $\{6Y, 1N\}$ and $\{3Y, 4N\}$. Compute $H(S)$, $H(S_v)$, and Gain(S, A) in bits. Should A be chosen over B with Gain 0.05?

E8.3 **Overfitting.** A depth-10 tree gets 99% train and 62% test accuracy. Name the phenomenon and give two remedies. What does Fig. 8.1 predict if depth goes to 15?

E8.4 **Perceptron trace.** Run the perceptron ($\eta = 1$, $w = [0, 0], b = 0$) on $(1, 1) \to 1$, $(-1, 0) \to 0$, $(0.5, 0.5) \to 1$ in order. Show each update and final w, b.

E8.5 **XOR and depth.** Prove no single perceptron separates XOR. Sketch a 2-hidden-unit network that does: give weights for OR and NAND hidden units and an AND output. Why is non-linearity essential?